GE-FANUC SNP DRIVER

Filename	SNP.DLL
Manufacturer	General Electric
Devices	90-30 e 90-70 PLCs Series
Protocol	SNP
Version	1.03
Latest update	DRV 01/25/2008 - DOC 02/26/2008
Platform	Win32
Dependencies	No dependencies

Introduction

The protocol implemented in the GE-Fanuc driver is SNP. Therefore, all devices that support this protocol can use this driver (Series 90-30 and 90-70 PLCs). The driver supports communication with PLCs in a network, since each PLC has a different ID.

Preparing the Equipment

Connection diagram of PC (Elipse) and GE-Fanuc

The device used in the tests had:

- A 10-SLOT BASE (IC693CHS391D);
- A SERIES 90-30 POWER SUPPLY (IC693PWR321L);
- A SERIES 90-30 CPU (IC693CPU331);
- A mini-converter RS-232 to RS-422 / RS-485 (IC690ACC900);
- A serial cable (IC690CBL702).

Follow these steps to assemble the device:

1) Connect the POWER SUPPLY and the CPU 90-30 in the rack (10-SLOT BASE) as explained in the Series 90-30 Programmable Controller Installation Manual;
2) Connect the mini-converter to the POWER SUPPLY serial port;
3) Connect the serial cable to the mini-converter and to the RS-232 computer port;
4) Connect the POWER SUPPLY cable to the energy supply. The PWR LED must be ON;
5) Program the PLC and change its ID using the LM90 software. If LM90 throws a TIMEOUT error, there are errors in the device installation. Check the serial port used by LM90 to communicate;
6) Create the GEFANUC.ID file (must be created in the same folder of the application) containing the ID programmed in LM90;
7) Configure the serial port, baud rate (usually 19200), timeout (30 is enough) and CPU type (90-30 or 90-70) using from P1 to P4 driver parameters;
8) Configure your application.

Memory Types

The GE-Fanuc driver supports the following memory types:

Table 1 - Memory Types

N2/B2	Size	Memory Type
0	Bit	Discrete Inputs (%I)
1	Byte	
2	Bit	Discrete Outputs (%Q)
3	Byte	
4	Bit	Discrete Temporaries (%T)
5	Byte	
6	Bit	Discrete Internals (%M)
7	Byte	
8	Bit	%SA Discretes
9	Byte	
10	Bit	%SB Discretes
11	Byte	
12	Bit	%SC Discretes
13	Byte	
14	Bit	%S Discretes
15	Byte	
16	Bit	Genius Global Data (%G)
17	Byte	
18	Word	Analog Inputs (%AI)
19	Word	Analog Outputs (%AQ)
20	Word	Registers (%R)
51	Float	
100-103	DWord	

DWORD types

Four types of *DWORD* were implemented for the registers:

100: unsigned inverted *DWORD*

101: unsigned *DWORD*

102: inverted signed *DWORD*

103: signed *DWORD*

DWORD numbers are composed of 32 bits, and represent two consecutive registers in the PLC memory. The difference between the 4 types of *DWORD* refers to the sequence that the registers are read and the handling of the value signal. The inverted types (100 and 102) correspond to *(%Rn * 65536) + %Rn+1*. The normal types (*101* and *103*) corresponds to *(%Rn+1 * 65536) + %Rn*.

For example, suppose that a *DWORD* is stored in the *%R6* and *%R7* registers, and each register has the following value: *%R6 = 40000* and *%R7 = 1*.

The value returned when reading *DWORD %R6* is:

- inverted unsigned *DWORD*: *%R6 * 65536 + %R7 = 2621440001*
- normal unsigned *DWORD*: *%R7 * 65536 + %R6 = 105536*
- inverted signed *DWORD*: *%R6 * 65536 + %R7 = -1673527295*
- normal signed *DWORD*: *%R7 * 65536 + %R6 = 105536*

After driver installation, the GEFANUC.ID file must exist in the current folder. This file creates a relationship between each PLC ID on the network with a number. This number will be used to identify the PLC in Elipse SCADA.

The GEFANUC.ID file format is the following:

```
num1 = id1 //Comments start with '//'and go till the end of the line
num2 = id2
...
numN = idN
```

Where *numN* might be a number from 0 to 49 (inclusive) and *idN* is the PLC ID.

For example, suppose we have 6 PLCs in a network with the following IDs: *ABCDE0*, *DEBA01*, *DEBA02*, *A00001*, *AB* and *506881*.

The GEFANUC.ID file for the example above must be:

```
// network configuration file
 0 = ABCDE0
 2 = DEBA01
 3 = DEBA02
 4 = A00001
48 = AB
49 = 506881
```

If the file has errors, or if it does not exist, the driver will not be installed on memory. It is important to say that unused numbers (in the example above: 1 and from 5 to 47) will have a null ID.

Driver settings [P] parameters

P1 Serial port number (1 = COM1, 2 = COM2, 3 = COM3, 4 = COM4, ..., n = COMn).

P2 IMH + CPU Type + Baud Rate (see Table 2).

P3 *Break* (in thousandth of seconds). Configures how long the driver will wait for a break recognition.

P4 *Timeout* (in hundredth of seconds). Configures how long the driver will wait for an answer from the PLC before generating a communication error.

Table 2 - IMH, CPU Type and Baud Rate

Value	IMH
0	Does not use IMH
1000	Uses IMH
Value	**CPU Type**
0	PLC 90-30
100	PLC 90-70
Value	**Baud Rate (bps)**
0	110
1	300
2	600
3	1200
4	2400
5	4800
6	9600
7	19200 (*normally used)

On P2 parameter, the other serial port parameters are: 8 data bits, 1 stop bit and odd parity. For CPUs 90-70, without using IMH, add 100 to this parameter. For example, to communicate at 9600 bps using a CPU 90-70 without IMH, the value must be *0 + 100 + 6 = 106*.

On P3 parameter, try to set this value always 10ms greater than the greatest SCAN time among the PLCs in the network. If this value is too high, the communication can become slow; if it is too low communication errors can occur. Check, by try and error, the lowest P3 value where communication errors do not happen.

The SCAN time is determined by the PLC program size.

WARNING: If P3 is under 5ms, it is considered as if it were 5ms.

Examples:

a) To communicate at 9600 bps using COM3, a timeout of 1 second and 5ms break:

p1 = 3 p2 = 6 p3 = 0 p4 = 100

b) To communicate at 19200 bps using COM1, 50ms break and a timeout of 20 hundredth of seconds

p1 = 1 p2 = 7 p3 = 50 p4 = 20

[N] Parameters of PLC tags

N1 PLC number (from 0 up to 49, as configured in the GEFANUC.ID file).

N2 Memory type (see Table 1).

N3 Address of the position to be read (starting at 1).

N4 Not used.

Examples:

a) To read the *%SB80* bit of the PLC with ID *AB*

```
n1 = 48 (48 = AB, as configured in GEFANUC.ID)
n2 = 10 (10 = %SB bit mode)
n3 = 80 (read bit 80)
n4 = 0
```

b) To read the register *%R594* of the PLC with ID *ABCDE0*

```
n1 = 0 (0 = ABCDE0, as configured GEFANUC.ID)
```

```
n2 = 20 (20 = %R, word mode)
n3 = 594 (register number to read)
n4 = 0
```

Modem parameterization

N1 -1 (minus 1).

N2 Modem command (see Table 3).

N3 Not used.

N4 Not used.

Table 3 - Modem Commands

N2	Function or Value	R/W?	Type
0	Telephone number	R/W	(String).
1	Dial	W	(Command Write).
2	Modem status	R	(String).
3	Carrier	R	(Bool).
4	Escape	W	(Command Write).
5	Baud Rate	R	(Double).

[B] Parameters of block tags

B1 PLC number (from 0 up to 49, as configured in GEFANUC.ID file).

B2 Memory type (see Table 1).

B3 Address of the first position to be read (starting at 1).

B4 Not used.

The number of positions to be read will be determined by the number of elements that build the Tag Block, therefore, each element corresponds to a memory register of the PLC, based on the starting position defined in the B3 parameter.

The maximum size of a Tag Block will be limited to a set of up to 100 elements (a value for each memory register), and for the types in *DWORD* and *FLOAT* format, the maximum size of a Tag Block will be limited to a set of up to 50 elements (a value for each two memory registers).

Examples:

a) To read the variables from *%G10* to *%G50* (inclusive) in *BYTE* mode of a PLC with ID *DEBA01* (using GEFANUC.ID as the example), use a Tag Block with 41 elements.

```
b1 = 2 (2 = DEBA01, as configured in GEFANUC.ID)
b2 = 17 (17 = %G, byte mode)
b3 = 10 (start in %G10)
b4 = 0
```

b) To read the variables from *%R10* to *%R50* (inclusive + *%R51*) in *FLOAT* mode of a PLC with ID *DEBA01* (using GEFANUC.ID as the example), use a Tag Block with 21 elements.

```
b1 = 2 (2 = DEBA01, as configured in GEFANUC.ID)
b2 = 51 (51 = %R, float mode)
b3 = 10 (start in %R10)
```

```
b4 = 0
```

c) To read 60 bits of a PLC with ID *506881* in *BIT* mode, starting at the address *%SA5*, use a Tag Block with 60 elements.

```
b1 = 49 (49 = 506881, as configures in GEFANUC.ID)
b2 = 8 (8 = %SA, bit mode)
b3 = 5 (start in %SA5)
b4 = 0
```

NOTE: In memory types that allow access by bit as well as by byte, the bit number is counted from the first memory position, where bit 1 is the less significant bit from byte 1.

```
+---------------+---------------------+----------------------+
|8|7|6|5|4|3|2|1|16|15|14|13|12|11|10|9|24|23|22|21|20|19|18|17|
+---------------+---------------------+----------------------+
         byte 1             byte 2                 byte 3
```

This way, the less significant bit from byte *N* is the bit *(N – 1) * 8 + 1*. The less significant bit from byte 10 is equal to *(10 – 1) * 8 + 1 = 73*. For example, the *%SB10* byte is formed by bits:

```
%SB80 %SB79 %SB78 %SB77 %SB76 %SB75 %SB74 %SB73
most                                      less
significant                         significant
```

Release history

Version	Date	Author	Comments
1.03	01/25/2008	C. Mello	- Adjustments for writing on Tag Block elements for *DWORD* and *FLOAT* types. (Case 9131).
1.02	02/26/2007	C. Mello	- Adjustments for manipulating *DWORD* data type. (Case 7845).
1.01	10/20/2004	C. Mello	- Implementation of individual reading of Tag Block Element. - Substitution of Log Date & Time option for Log Timestamp in Log page, in the Extras dialog box. The Log Date & Time option will be used by default for the driver. - General revision of all source codes.
1.00			- All releases before revision control.